



PROJECT AND TEAM INFORMATION

Project Title

WALnut: Concurrent, Deadlock-Free Multidimensional Database Engine

Student / Team Information

Team Name: Team #	<i>The Acid Mechanics</i> <i>T038</i>
Team member 1 (Team Lead)	<i>Suyal, Vinayak – 240211122</i> <i>suyalvinayak22@gmail.com</i> 
Team member 2	<i>Singh Rawat, Sushmit– 24021051</i> <i>7838sushmit@gmail.com</i> 
Team member 3	<i>Rawat, Kanak–240211907</i> <i>kavyarawatkavya831@gmail.com</i> 
Team member 4	<i>Jugran, Suhavi – 240211640</i> <i>suhavijugran@gmail.com</i> 

PROPOSAL DESCRIPTION

Motivation

Today's applications require databases in order to store, organize and process huge amount of data. This was our motivation to develop a Concurrent and Deadlock-free Relational DB Engine.

There will be fundamental DBMS concepts like ER Modeling, Primary Key, Foreign Key, Functional Dependencies, Normalization up to 3NF/BCNF, SQL, Relational Algebra and Indexing.

However the greatest challenge occurs when multiple users conduct transactions simultaneously. Concurrent transactions can clash on the same rows, tables, memory pages and disk space resulting in inconsistent data, blocking, resource conflicts or deadlocks. In order to solve the above problem the project will integrate transaction management of DBMS with operating system concepts like Threads, Synchronization, Memory Management and Resource Allocation.

*There will be implementation of **ACID Transactions, Strict 2 PL, Serializability and Deadlock Avoidance using Banker's Algorithm**. A custom made buffer pool, page replacement scheme, storage management and disk scheduling algorithms will handle system resources. **WAL Logging and Checkpointing** will allow crash recovery.*

By doing this project, we hope to understand and show how concepts of DBMS and OS can be used together to create a good database system.

State of the Art / Current solution

The current relational database systems solve most of these challenges in production. Database systems such as PostgreSQL and SQLite offer transaction management, concurrency control, indexing, page caching, and crash recovery. For example, SQLite employs a page-based architecture with page cache, locking mechanisms, and B-tree data storage, where its WAL mode enables the logging of changes to a separate file to be copied to the database during the checkpoints.

The modern database engines also provide mechanisms for preserving ACID properties, isolation, locking, and recovery so that multiple users can access and modify the data. For example, PostgreSQL employs the WAL mechanism to guarantee logging of changes before data pages are written back so that recovery after failure is possible.

Yet, these systems are very complex, and most of the internal mechanisms are abstracted from the user. As a result, it is hard for a student to see the interaction between query processing, locking, buffer management, synchronization, deadlocks, storage, and recovery.

Our project aims to tackle this challenge with an educational and experimental approach by developing a relational database engine with a simple web interface/application and measuring its performance and behavior under concurrent workloads.

Project Goals and Milestones

Project Objectives

- *Working Database: Creation of a very basic DBMS that can create, search, update, and handle relational data via SQL queries.*
- *Fast Data Access: Increase speed of frequently used SELECTs and JOINs by using relational algebra and indexing.*
- *Safe Multithreaded Concurrency: Perform multiple transactions concurrently without any possibility of data corruption or inconsistency.*
- *Database Operations vs OS: Study effect of threads, synchronization, memory management, deadlock prevention and disk scheduling on database operations.*
- *Self Storage Manager: Perform all file, page and memory management operations for database by implementing storage manager and buffer pool.*
- *Data Protection: Protect data from being lost in case of a failure through WAL, checkpointing and crash recovery.*
- *Monitoring: Provide a web interface where queries, transactions, locks and performance can be monitored and analyzed easily.*

Key Milestones

- *Planning: Requirements analysis, research, ER diagram and design.*
- *Database Initialization: Database schema, normalization, storage and basic database operations.*
- *Queries Indexing: SQL parser, relational algebra, query processing and B+ tree/ hashing indexing.*
- *Transactions: ACID properties, locks, strict 2PL and serializability.*
- *Integration with OS: Threads, synchronization, buffer and disk scheduling.*
- *Recovery: Deadlock prevention, Banker's algorithm, WAL and checkpointing.*

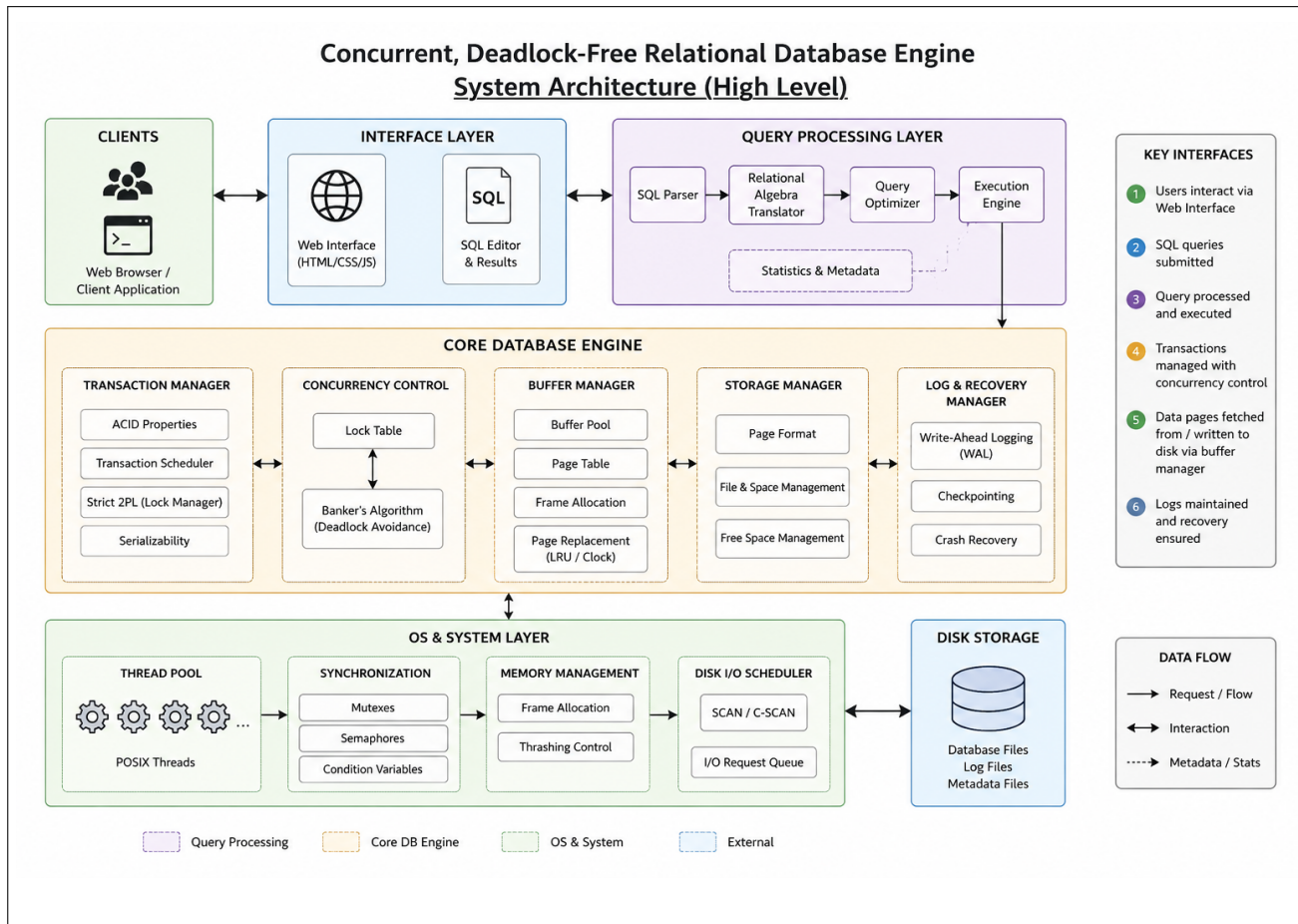
Project Approach

The project will be developed step-by-step, starting with the basic database structure and gradually adding concurrency, OS-level resource management and recovery features. The main focus will be on building the database engine ourselves rather than depending on an existing DBMS for query execution.

- **Database Design:** Create ER model, identify the functional dependencies and normalize the schema up to 3NF/BCNF. The data will be organized into relational tables with appropriate keys and relationships.
- **Query Engine:** A lightweight SQL-subset parser and convert queries into relational algebra operations such as *SELECT*, *PROJECT* and *JOIN*. B+ Tree or Hash indexing will be added to improve data retrieval.
- **Transaction Concurrency Layer:** Implement ACID transactions, Strict 2PL, locking and serializability. A custom thread pool with mutexes, semaphores and condition variables will allow multiple transactions to execute safely.
- **OS Storage Layer:** Develop a buffer pool, page management and page-replacement mechanism for memory management. Banker's Algorithm will be used for deadlock avoidance while SCAN/C-SCAN will manage disk I/O requests.
- **Recovery Interface:** Implement Write-Ahead Logging (WAL), checkpointing and crash recovery. A web interface/application will allow users to submit SQL queries and observe query execution, transactions, resource usage and system performance.
- **Testing:** Run different concurrent workloads and compare performance using query latency, throughput, cache hits, memory usage, page faults and disk I/O.

The engine will primarily be developed in C/C++ using POSIX threads and file handling.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The project shall develop a concurrent process management and database system showing how Operating System and DBMS concepts interact in multi-user settings.

- The development of concurrent processes/threads for performing several tasks at once.
- Synchronization and mutual exclusion with the use of locks/semaphores preventing race conditions.
- A database that allows to store and manage shared resources and data.
- Managing of concurrent database transactions with maintenance of data integrity.
- Illustration of deadlock prevention/detection and resource management techniques.
- A user interface allowing to observe processes, transactions, and overall system performance.
- Evaluation of performance in terms of concurrency, responsiveness, and resource consumption.
- Comprehensive documentation containing description of architecture, algorithms, database schema,
- implementation and testing of the project.
- The resulting product shall show a real-life example of how to access shared resources and database records concurrently and safely.

Assumptions

The system shall allow multiple users/processes to use shared resources simultaneously.

- *Every user will have proper authentication with appropriate permission.*
- *The database shall be available and configured in an appropriate manner in the whole system operation process.*
- *Simultaneous transactions can read/write the same data and hence there is a need for synchronization.*
- *The number of simultaneous processes shall be within the capacity of the system.*
- *Normal system failures like abrupt system shutdown or network failure are initially excluded.*
- *Standard OS notions like process scheduling, synchronization, and deadlock management will be*

References

1. *SQLite Documentation, "Architecture of SQLite."*
<https://www.sqlite.org/arch.html>
2. *PostgreSQL Documentation, "Write-Ahead Logging (WAL)."*
<https://www.postgresql.org/docs/current/wal-intro.html>
3. *MySQL Documentation, "InnoDB Locking and Transaction Model."*
<https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-transaction-model.html>
4. *IBM Documentation, "POSIX thread programming."*
<https://www.ibm.com/docs/en/zos/2.5.0?topic=services-thread-programming>